

Algoritmo del Simplexso Primale

Step1. Inizializzazione:

Definisce una soluzione base ammissibile

$\mathbf{x} = [\mathbf{x}_B, \mathbf{x}_N] = [\mathbf{B}^{-1}\mathbf{b}, \mathbf{0}] = [\bar{\mathbf{b}}, \mathbf{0}]$ di costo $z = \mathbf{c}_B\mathbf{x}_B = \mathbf{c}_B\mathbf{B}^{-1}\mathbf{b}$.

Step2. Pricing:

Calcola $\mathbf{w} = \mathbf{c}_B\mathbf{B}^{-1}$, che equivale a risolvere $\mathbf{wB} = \mathbf{c}_B$.

Calcola i costi ridotti $\mathbf{w}\mathbf{a}_j - c_j$ per le variabili non-base $j \in N$ e determina:

$$\mathbf{w}\mathbf{a}_k - c_k = \max_{j \in N} \{ \mathbf{w}\mathbf{a}_j - c_j \}$$

Step3. Condizioni di ottimalità:

Se $\mathbf{w}\mathbf{a}_k - c_k < 0$, allora STOP la soluzione è *ottima*.

Step4. La variabile k è candidata a entrare in base:

Calcola $\mathbf{y}^k = \mathbf{B}^{-1}\mathbf{a}_k$, che equivale a risolvere $\mathbf{B}\mathbf{y}^k = \mathbf{a}_k$.

Se $\mathbf{y}^k \leq \mathbf{0}$, allora STOP la soluzione è *illimitata*.

Algoritmo del Simplexso Primale (2)

Step5. Rapporto minimo:

Calcola il valore da assegnare a x_k :

$$x_k = \frac{\bar{b}_r}{y_r^k} = \min \left\{ \frac{\bar{b}_i}{y_i^k} : y_i^k > 0, i = 1, \dots, m \right\}$$

La variabile x_r esce dalla base e x_k entra al suo posto.

Aggiorna $\mathbf{B}$, $\mathbf{N}$ e la soluzione base $\mathbf{x} = [\mathbf{x}_B, \mathbf{x}_N] = [\bar{\mathbf{b}}, \mathbf{0}]$.

Ritorna allo Step 2.

Relazione tra Primale e Duale (2)

- Se il primale non ha soluzioni ammissibili, i.e. $X = \emptyset$, allora il duale o non ha soluzioni ammissibili o ha una soluzione ottima non limitata.
- Possiamo riassumere tutti i possibili casi nella seguente tabella:

		D		
		Ottimo	Non Amm.	Illim.
P	Ottimo	X		
	Non Amm.		X	X
	Illim.		X	

Il Metodo del Simpleso Formato Tableau (3)

In una versione di maggiore dettaglio il “*tableau*” è il seguente:

		$\mathbf{x_B}$					$\mathbf{x_N}$					RHS	
z	z	1	0	...	0	...	0	$\mathbf{wa}_{m+1} - c_{m+1}$	...	$\mathbf{wa}_{m+j} - c_{m+j}$	...	$\mathbf{wa}_n - c_n$	$\mathbf{c_B B^{-1} b}$
$\mathbf{x_B}$	0	1	...	0	...	0	y_1^{m+1}	...	y_1^j	...	y_1^n	$\bar{b}_1$	
	...	...	...	...	...	...	...	...	...	...	...	...	
	0	0	...	1	...	0	y_i^{m+1}	...	y_i^j	...	y_i^n	$\bar{b}_i$	
	...	...	...	...	...	...	...	...	...	...	...	...	
	0	0	...	0	...	1	y_m^{m+1}	...	y_m^j	...	y_m^n	$\bar{b}_m$	

- Come si può notare il tableau contiene tutte le informazioni necessarie per l'esecuzione dell'algoritmo del simpleso.
- L'operazione base è il *pivoting*. Che permette a una nuova variabile di entrare in base e di aggiornare *correttamente* tutte le informazioni nel tableau (costi ridotti, valore variabili base, etc.)
- Si illustra l'utilizzo del tableau con un esempio.

Come determinare una base iniziale: Metodo 2-Fasi

- Sia dato un problema della seguente forma:

$$\begin{aligned} (P) \quad z_P = \min \quad & \mathbf{c}\mathbf{x} \\ \text{s.t.} \quad & \mathbf{A}\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

- Nell'ipotesi che $\mathbf{b} \geq \mathbf{0}$, si possono aggiungere m variabili $\mathbf{x}_A$, dette *artificiali*, alle n variabili originarie, e risolvere il seguente problema:

$$\begin{aligned} (P') \quad z_{P'} = \min \quad & \mathbf{1}\mathbf{x}_A \\ \text{s.t.} \quad & \mathbf{A}\mathbf{x} + \mathbf{I}\mathbf{x}_A = \mathbf{b} \\ & \mathbf{x}, \mathbf{x}_A \geq \mathbf{0} \end{aligned}$$

dove $\mathbf{I}$ è la matrice identità di ordine m e $\mathbf{1} = \{1, 1, \dots, 1\}$, è un vettore di m componenti tutte pari a 1.

Come determinare una base iniziale: Metodo 2-Fasi (2)

- In questo caso i problemi P e P' non sono equivalenti.
- Risolvere il problema P' serve solo a determinare una soluzione base ammissibile per il problema P .
- Sia $(\mathbf{x}^*, \mathbf{x}_A^*)$ la soluzione ottima del problema P' di valore $z_{P'}$. Si possono presentare tre casi:
 - $z_{P'} > 0$
 $\Rightarrow$ **il problema P non ha una base ammissibile;**
 - $z_{P'} = 0$ e nessuna variabile artificiale è in base
 $\Rightarrow$ **il problema P ha una base ammissibile;**
 - $z_{P'} = 0$ e almeno una variabile artificiale è in base
 $\Rightarrow$ **il problema P ha una base ammissibile, ma bisogna *estrarla*.**

Algoritmo del Simplexso Duale

Step1. Inizializzazione:

Sia $\mathbf{B}$ una base duale ammissibile: $\mathbf{w}\mathbf{a}_j - c_j = \mathbf{c}_B\mathbf{B}^{-1}\mathbf{a}_j - c_j \leq 0$.

Step3. Determinazione riga r :

$\bar{b}_r = \min\{\bar{b}_i : i = 1, \dots, m\}$.

Se $\bar{\mathbf{b}} \geq \mathbf{0}$, allora STOP perché la soluzione è primale ammissibile e quindi *ottima*.

Step4. Determinazione colonna k :

Applica il rapporto minimo:

$$\frac{\mathbf{w}\mathbf{a}_k - c_k}{y_r^k} = \min \left\{ \frac{\mathbf{w}\mathbf{a}_j - c_j}{y_r^j} : y_r^j < 0, j = 1, \dots, n \right\}$$

Se $y_r^j \geq 0$, per ogni $j = 1, \dots, n$, allora STOP perché il duale è illimitato e la soluzione ammissibile del primale non esiste.

Algoritmo del Simplexso Duale (2)

Step3. Pivoting:

Svolgi un operazione di pivoting sull'elemento (r, k) .

Ritorna allo Step 2.

NOTE:

- Il valore della funzione obiettivo $z = \mathbf{c}_B \mathbf{x}_B = \mathbf{c}_B \mathbf{B}^{-1} \mathbf{b}$ è un lower bound.
- Durante l'esecuzione del simplexso duale il valore della funzione obiettivo cresce in modo monotonicamente non decrescente, perchè:

$$\mathbf{c}_B \bar{\mathbf{b}} = \mathbf{c}_B \bar{\mathbf{b}} - \frac{\bar{b}_r}{y_r^k} (\mathbf{w} \mathbf{a}_k - c_k).$$

Tagli di Gomory

- Per semplicità avevamo ipotizzato che la base fosse costituita dalle prime m colonne. Nel caso generale, il Taglio di Gomory relativo alla variabile in base frazionaria in corrispondenza della riga i può essere riscritto come:

$$-\sum_{j \in N} F_i^j x_j \leq -F_i \quad (9)$$

dove N è l'insieme delle colonne non in base.

- Se risolviamo il problema con il Simplexso, quando aggiungiamo il nuovo vincolo al problema non è necessario ripetere l'algoritmo dall'inizio, ma possiamo ripartire dalla base corrente.
- Se le variabili attualmente incluse sono n , il taglio di Gomory corrisponde a una nuova riga da aggiungere al tableau:

$$-\sum_{j \in N} F_i^j x_j + x_{n+1} = -F_i \quad (10)$$

Algoritmo di Bellman-Ford

Algoritmo di Bellman-Ford

Require: Grafo orientato;

Ensure: Cammini minimi da s a $V \setminus \{s\}$ definiti da $pred[j]$, $\forall j \in V \setminus \{s\}$;

// Inizializzazione

for $j = 1$ to n **do**

$d[j] = \infty$; $pred[j] = -1$;

end for

$d[s] = 0$;

// Controlla gli archi per $n - 1$ iterazioni

for $k = 1$ to $n - 1$ **do**

for $(i, j) \in A$ **do**

if $(d[j] > d[i] + c_{ij})$ **then**

$d[j] = d[i] + c_{ij}$; $pred[j] = i$;

end if

end for

end for

Algoritmo di Bellman-Ford (2)

```
// Controlla se ci sono cicli di costo negativo
for  $(i,j) \in A$  do
  if  $(d[j] > d[i] + c_{ij})$  then
    Il grafo contiene cicli di costo negativo;
  end if
end for
```

Osservazioni:

- Se nel ciclo principale non vengono trovate condizioni di ottimalità violate, allora nessuna distance label $d(i)$ verrà aggiornata.
- Se nessuna distance label verrà modificata, allora anche nell'iterazione successiva nessuna condizione di ottimalità sarà violata.
- L'algoritmo può essere ulteriormente migliorato modificando il ciclo principale per permettere un'uscita anticipata quando nessuna distance label potrà essere ulteriormente modificata.

Algoritmo di Bellman-Ford (3)

```
...  
//Controlla gli archi per  $n - 1$  iterazioni  
for  $k = 1$  to  $n - 1$  do  
     $update = False$ ;  
    for  $(i, j) \in A$  do  
        if  $(d[j] > d[i] + c_{ij})$  then  
             $d[j] = d[i] + c_{ij}$ ;  
             $pred[j] = i$ ;  
             $update = True$ ;  
        end if  
    end for  
    if  $(update = False)$  then  
        Esci dal ciclo;  
    end if  
end for  
...
```

Algoritmo di Dijkstra: Versione Migliorata

Algoritmo di Dijkstra (versione $O(n^2)$)

Require: Grafo orientato connesso con costi $\{c_{ij}\}$ non-negativi;

Ensure: Cammini minimi da s a $V \setminus \{s\}$ definiti da $pred[j]$, $\forall j \in V \setminus \{s\}$;

// Inizializzazione

for $j = 1$ to n **do**

$flag[j] = 0$; $pred[j] = s$; $L[j] = c_{sj}$;

end for

$flag[s] = 1$; $L[s] = 0$;

for $k = 1$ to $n - 1$ **do**

 // Individua $h = \operatorname{argmin}\{L[j] : j \notin S\}$

$min = +\infty$;

for $j = 1$ to n **do**

if $(flag[j] = 0)$ **and** $(L[j] < min)$ **then**

$min = L[j]$; $h = j$;

end if

end for

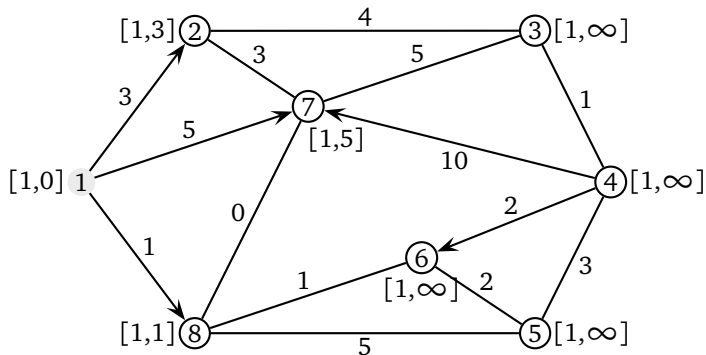
Algoritmo di Dijkstra: Versione Migliorata (2)

```
// Aggiorna  $S = S \cup \{h\}$   
 $flag[h] = 1$ ;  
// Aggiorna  $L[j]$  e  $pred[j]$  per ogni  $j \notin S$   
for  $j = 1$  to  $n$  do  
    if ( $flag[j] = 0$ ) and ( $L[h] + c_{hj} < L[j]$ ) then  
         $L[j] = L[h] + c_{hj}$ ;  
         $pred[j] = h$ ;  
    end if  
end for  
end for
```

NOTA: Se si è interessati a calcolare i cammini minimi dal vertice s a un sottoinsieme di vertici $T \subseteq V$ (eventualmente contenente un solo vertice t , i.e., $T = \{t\}$), possiamo fermare l'Algoritmo di Dijkstra non appena le etichette di tutti i vertici di T sono in S , quindi permanenti.

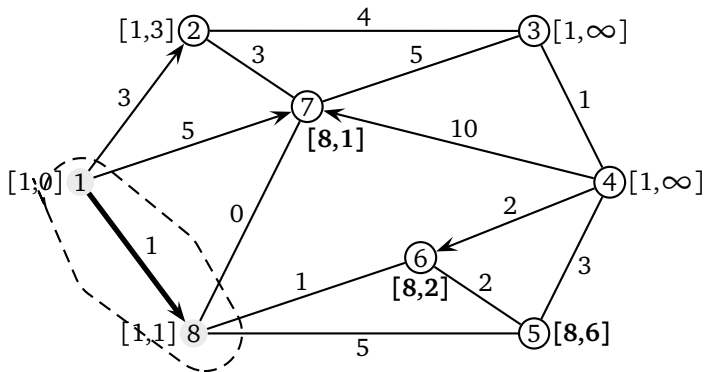
Algoritmo di Dijkstra: Esempio

Inizializzazione $s = 1$ (etichette $[pred[j], L[j]]$ sui vertici)



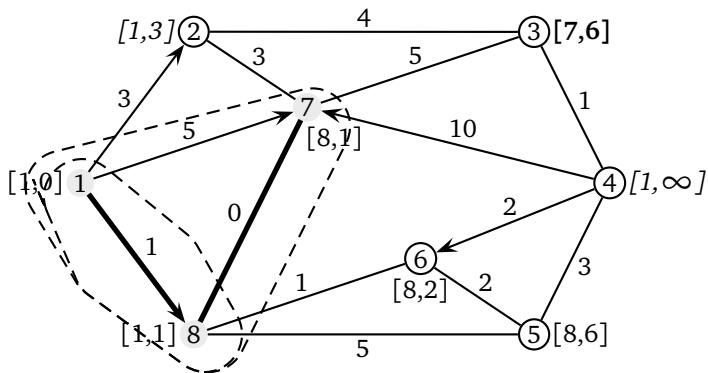
Algoritmo di Dijkstra: Esempio (2)

Vertice scelto 8



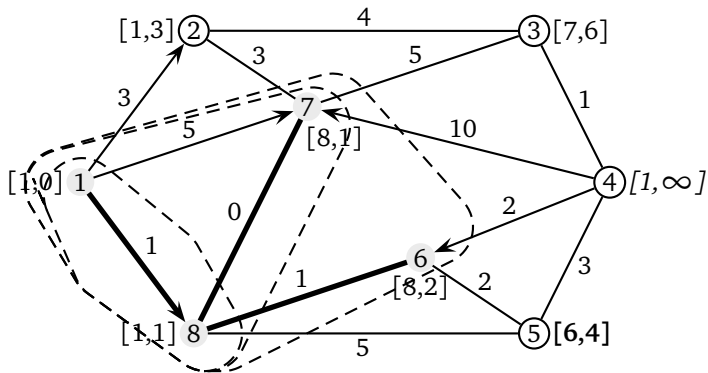
Algoritmo di Dijkstra: Esempio (3)

Vertice scelto 7



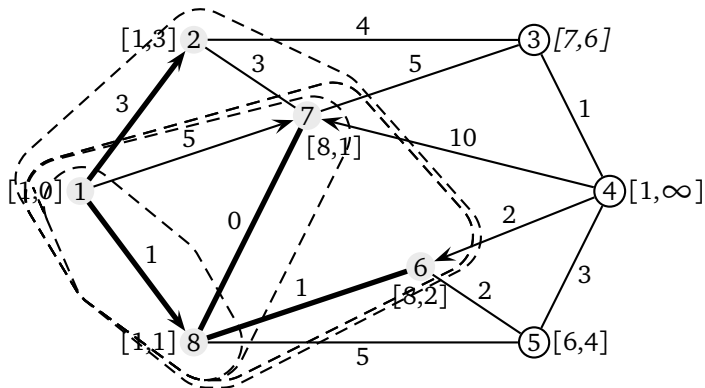
Algoritmo di Dijkstra: Esempio (4)

Vertice scelto 6



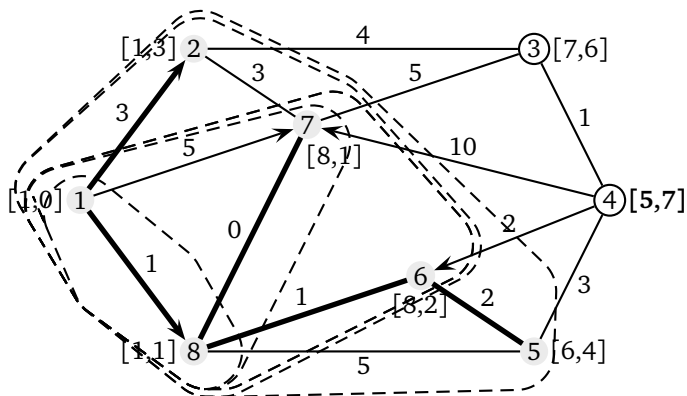
Algoritmo di Dijkstra: Esempio (5)

Vertice scelto 2



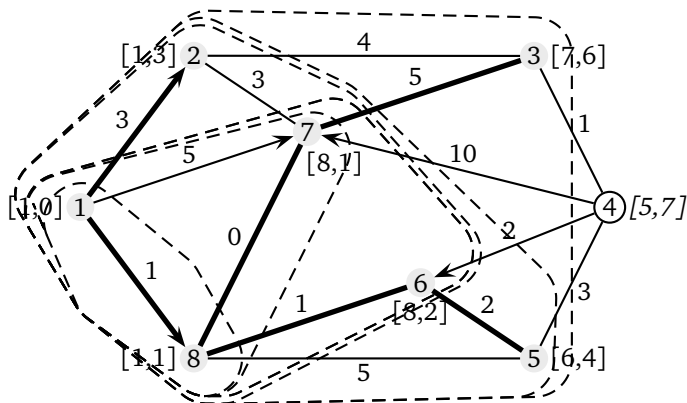
Algoritmo di Dijkstra: Esempio (6)

Vertice scelto 5



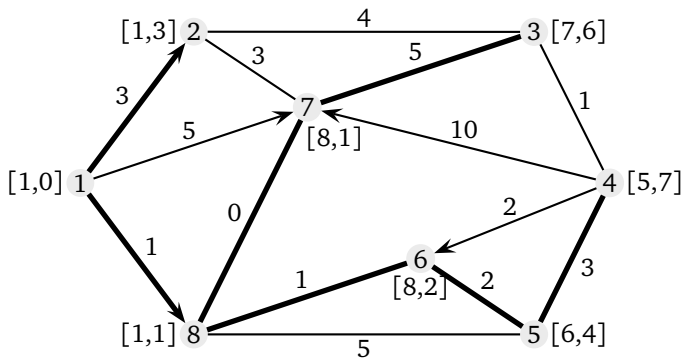
Algoritmo di Dijkstra: Esempio (7)

Vertice scelto 3



Algoritmo di Dijkstra: Esempio (8)

Vertice scelto 4



Algoritmo di Floyd-Warshall

Algoritmo di Floyd-Warshall

Require: Grafo orientato definito dalla matrice dei costi $[c_{ij}]$;

Ensure: Matrici $[u_{ij}]$ e $[pred[i,j]]$;

// Inizializzazione

for $i = 1$ to n **do**

for $j = 1$ to n **do**

$u_{ij} = c_{ij}$; $pred[i,j] = i$;

end for

end for

// Operazione triangolare su k

for $k = 1$ to n **do**

for $i = 1$ to n **do**

for $j = 1$ to n **do**

if $(u_{ik} + u_{kj} < u_{ij})$ **then**

$u_{ij} = u_{ik} + u_{kj}$; $pred[i,j] = pred[k,j]$;

end if

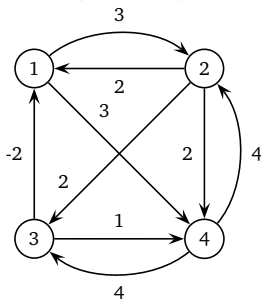
Algoritmo di Floyd-Warshall (2)

```
    end for
  end for
  for  $i = 1$  to  $n$  do
    if  $(u_{ii} < 0)$  then
      STOP, circuiti negativi;
    end if
  end for
end for
```

Nota: Ciascun valore u_{ij} calcolato all'iterazione k -esima dell'Algoritmo di Floyd-Warshall rappresenta il costo del cammino di costo minimo dal vertice i al vertice j usando come vertici *interni* al cammino i vertici dell'insieme $\{1, 2, \dots, k\}$.

Algoritmo di Floyd-Warshall: Esempio

Consideriamo il grafo seguente:



$$[c_{ij}] = \begin{bmatrix} 0 & 3 & \infty & 3 \\ 2 & 0 & 2 & 2 \\ -2 & \infty & 0 & 1 \\ \infty & 4 & 4 & 0 \end{bmatrix}$$

Inizializzazione:

$$u_{ij}$$

0	3	∞	3
2	0	2	2
-2	∞	0	1
∞	4	4	0

$$pred[i,j]$$

1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4

Algoritmo di Floyd-Warshall: Esempio (2)

Alla prima iterazione con $k = 1$ si ha:

$$u_{ij} = \text{Min} \{u_{ij}, (u_{i1} + u_{1j})\}.$$

u_{ij}					u_{ij}			
0	3	∞	3	$\Rightarrow$	0	3	∞	3
2	0	2	2		2	0	2	2
-2	∞	0	1		-2	1	0	1
∞	4	4	0		∞	4	4	0

$pred[i,j]$					$pred[i,j]$			
1	1	1	1	$\Rightarrow$	1	1	1	1
2	2	2	2		2	2	2	2
3	3	3	3		3	1	3	3
4	4	4	4		4	4	4	4

Ad esempio:

$$u_{32} = \text{Min} \{u_{32}, (u_{31} + u_{12})\} = \text{Min} \{\infty, (-2 + 3)\} = 1.$$

Algoritmo di Floyd-Warshall: Esempio (3)

Alla seconda iterazione con $k = 2$ si ha:

$$u_{ij} = \text{Min} \{u_{ij}, (u_{i2} + u_{2j})\}.$$

u_{ij}					u_{ij}			
0	3	∞	3	$\Rightarrow$	0	3	5	3
2	0	2	2		2	0	2	2
-2	1	0	1		-2	1	0	1
∞	4	4	0		6	4	4	0

$pred[i,j]$					$pred[i,j]$			
1	1	1	1	$\Rightarrow$	1	1	2	1
2	2	2	2		2	2	2	2
3	1	3	3		3	1	3	3
4	4	4	4		2	4	4	4

Ad esempio:

$$u_{13} = \text{Min} \{u_{13}, (u_{12} + u_{23})\} = \text{Min} \{\infty, (3 + 2)\} = 5.$$

Algoritmo di Floyd-Warshall: Esempio (4)

Alla terza iterazione con $k = 3$ si ha:

$$u_{ij} = \text{Min} \{u_{ij}, (u_{i3} + u_{3j})\}.$$

u_{ij}					u_{ij}			
0	3	5	3	$\Rightarrow$	0	3	5	3
2	0	2	2		0	0	2	2
-2	1	0	1		-2	1	0	1
6	4	4	0		2	4	4	0

$pred[i,j]$					$pred[i,j]$			
1	1	2	1	$\Rightarrow$	1	1	2	1
2	2	2	2		3	2	2	2
3	1	3	3		3	1	3	3
2	4	4	4		3	4	4	4

Ad esempio:

$$u_{21} = \text{Min} \{u_{21}, (u_{23} + u_{31})\} = \text{Min} \{2, (2 - 2)\} = 0.$$

Algoritmo di Floyd-Warshall: Esempio (5)

Alla quarta e ultima iterazione con $k = 4$ si ha:

$$u_{ij} = \text{Min} \{u_{ij}, (u_{i4} + u_{4j})\}.$$

u_{ij}					u_{ij}			
0	3	5	3	$\Rightarrow$	0	3	5	3
0	0	2	2		0	0	2	2
-2	1	0	1		-2	1	0	1
2	4	4	0		2	4	4	0

$pred[i,j]$					$pred[i,j]$			
1	1	2	1	$\Rightarrow$	1	1	2	1
3	2	2	2		3	2	2	2
3	1	3	3		3	1	3	3
3	4	4	4		3	4	4	4

Ad esempio:

$$u_{23} = \text{Min} \{u_{23}, (u_{24} + u_{43})\} = \text{Min} \{2, (2 + 4)\} = 2.$$

Condizioni di Ottimalità (3)

Teorema (Cut Optimality Conditions). L'albero di copertura $G_T = (V, T)$ è di costo minimo se e solo se soddisfa le seguenti condizioni di ottimalità: per ogni lato appartenente all'albero $e \in T$, $c_e \leq c_g$ per ogni lato g appartenente al taglio S e $V \setminus S$ generato dall'eliminazione del lato e .

Dimostrazione

(a) Se l'albero di copertura è di costo minimo, allora *deve* soddisfare le condizioni di ottialità.

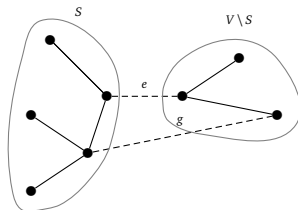
Se per un lato $e \in T$ accadesse che $c_e > c_g$ per un lato g appartenente al taglio S e $V \setminus S$ generato dall'eliminazione del lato e , allora basterebbe sostituire il lato e con g in T per ottenere un albero di copertura di costo più basso contraddicendo l'ipotesi iniziale.

Condizioni di Ottimalità (4)

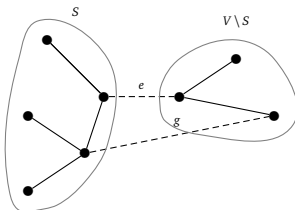
- (b) Se l'albero di copertura rispetta le condizioni di ottimalità, allora è un albero di copertura di costo minimo.

Supponiamo che l'albero di copertura T non sia quello di costo minimo e che quello ottimo sia T^* . Quindi, esiste un lato $e \in T$ che non è nell'albero di copertura di costo minimo, i.e., $e \notin T^*$.

Eliminare il lato e da T crea un taglio corrispondente ai due alberi così ottenuti, mentre aggiungere il lato e all'albero ottimo T^* crea un ciclo.



Condizioni di Ottimalità (5)



L'albero T soddisfa le condizioni di ottimalità, quindi $c_e \leq c_g$ per tutti i lati del taglio compreso il lato g che assieme al lato e crea un ciclo nell'albero ottimo T^* .

Però, se T^* è l'albero ottimo $c_e \geq c_g$, altrimenti potremo trovare un albero con costo più basso scambiando i lati.

Per cui, $c_e = c_g$. Se ripetiamo, questo processo per tutti i lati di T che non sono in T^* otterremo lo stesso risultato. Quindi, l'albero di copertura T è di costo minimo.

Condizioni di Ottimalità (6)

Teorema (Path Optimality Conditions). L'albero di copertura $G_T = (V, T)$ è di costo minimo se e solo se soddisfa le seguenti condizioni di ottimalità: per ogni lato g non appartenente all'albero (i.e., $g \notin T$), $c_g \geq c_e$ per ogni lato e contenuto nel path in T che collega gli estremi del lato g .

Dimostrazione

(a) Se l'albero di copertura è di costo minimo, allora *deve* soddisfare le condizioni di ottialità.

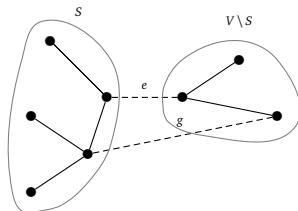
Se e è un lato del path in T che collega gli estremi del lato $g \notin T$ e $c_e > c_g$, potremo sostituire il lato e con g ottenendo un albero di copertura con un costo più basso contraddicendo l'ipotesi iniziale.

Condizioni di Ottimalità (7)

- (b) Se l'albero di copertura rispetta le condizioni di ottimalità, allora è un albero di copertura di costo minimo.

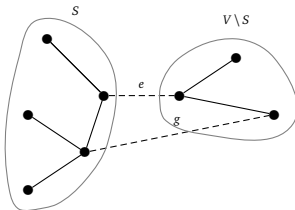
Dimostriamo che un albero T che soddisfa le *path optimality conditions* soddisfa anche le *cut optimality conditions*, così, per il teorema precedente, T è l'albero di copertura di costo minimo.

Sia e un qualsiasi lato appartenente all'albero T , e sia S e $V \setminus S$ il taglio generato dalla sua eliminazione. Per ogni lato g del taglio, nell'albero T ci sarà un unico path P che collega gli estremi di g .



Condizioni di Ottimalità (8)

Siccome e era l'unico lato del taglio che apparteneva a T (per costruzione), deve appartenere al path P .



Per ipotesi le condizioni di ottimalità sono valide, quindi $c_g \geq c_e$ per ogni lato e contenuto nel path in T che collega gli estremi del lato g . Siccome questa condizione deve essere valida per tutti i lati non appartenenti a T del taglio ottenuto eliminando un qualsiasi lato di T , allora deve soddisfare le *cut optimality conditions*.

Condizioni di Ottimalità (9)

Quindi l'albero T è quello di costo minimo.

NOTA: La dimostrazione del teorema dimostra anche l'equivalenza tra le *cut optimality conditions* e le *path optimality conditions*.

Algoritmo di Prim-Dijkstra (2^a versione)

Algoritmo di Prim-Dijkstra (2^a versione)

Require: Grafo non orientato $G(V, E)$ pesato con costi c_e , $\forall e \in E$;

Ensure: Albero di copertura di costo minimo T^* definito da $pred[j], j \in V$;

// Inizializza le strutture dati partendo da $S = \{1\}$

$flag[1] = 1$;

$pred[1] = 1$;

for $j = 2$ to n **do**

$flag[j] = 0$;

$L[j] = c_{1j}$;

$pred[j] = 1$;

end for

// Seleziona gli $n - 1$ lati dell'albero

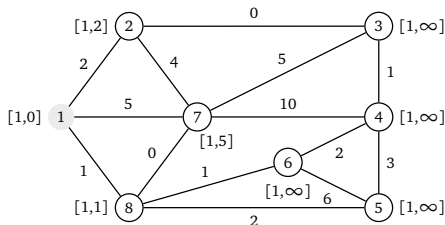
for $k = 1$ to $n - 1$ **do**

 // Sceglie il lato minimo in $\delta(S)$

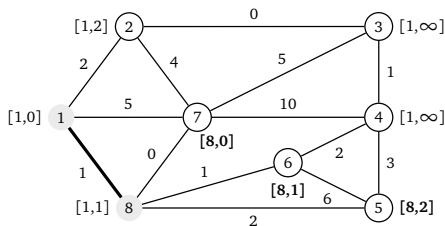
Algoritmo di Prim-Dijkstra (2^a versione) (2)

```
min =  $+\infty$ ;  
for  $j = 2$  to  $n$  do  
    if ( $flag[j] = 0$ ) and ( $L[j] < min$ ) then  
         $min = L[j]$ ;  $h = j$ ;  
    end if  
end for  
// Include il vertice  $h$  in  $S$  e aggiorna  $L[j]$  e  $pred[j]$  per ogni  $j \notin S$   
 $flag[h] = 1$ ;  
for  $j = 2$  to  $n$  do  
    if ( $flag[j] = 0$ ) and ( $c_{hj} < L[j]$ ) then  
         $L[j] = c_{hj}$ ;  $pred[j] = h$ ;  
    end if  
end for  
end for
```

Algoritmo di Prim-Dijkstra (2^a versione): Esempio

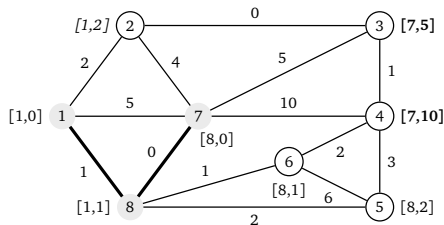


Inizializzazione
etichette $[pred[j], L[j]]$
sui vertici

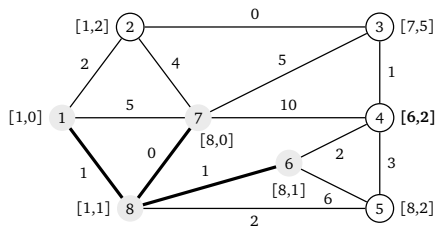


Scelto vertice 8

Algoritmo di Prim-Dijkstra (2^a versione): Esempio (2)

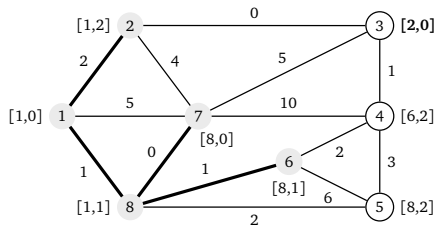


Scelto vertice 7

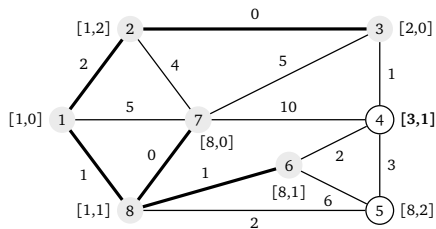


Scelto vertice 6

Algoritmo di Prim-Dijkstra (2^a versione): Esempio (3)

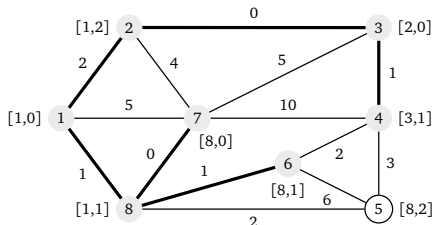


Scelto vertice 2

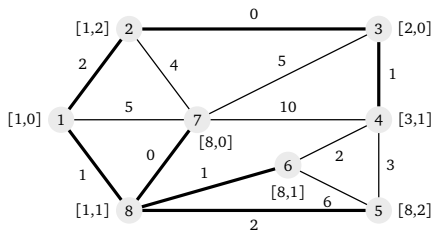


Scelto vertice 3

Algoritmo di Prim-Dijkstra (2^a versione): Esempio (4)



Scelto vertice 4



Scelto vertice 5
Il costo dello SST è 7

Algoritmo di Kruskal (6)

Algoritmo di Kruskal (2^a versione)

Require: Grafo non orientato $G(V, E)$ pesato con costi c_e , $\forall e \in E$;

Ensure: Albero di copertura di costo minimo T^* ;

// Inizializza le strutture dati e ordina i lati

Ordina i lati per costi crescenti ($E = \{e_1, \dots, e_m\} : c(e_i) \leq c(e_{i+1})$);

$k = 0$; $h = 0$;

// Ogni vertice rappresenta una componente distinta

for $i = 1$ to n **do**

$comp[i] = i$;

end for

// Costruisci l'albero di copertura di costo minimo

while $(k < n - 1)$ **and** $(h < m)$ **do**

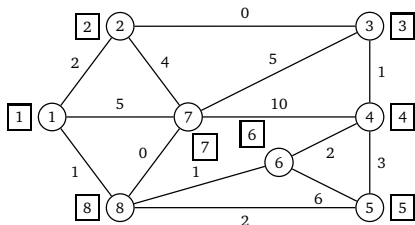
 Seleziona il lato $e_h = \{i, j\}$;

$C1 = comp[i]$; $C2 = comp[j]$;

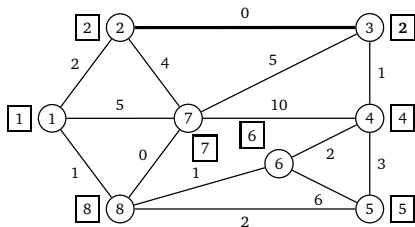
Algoritmo di Kruskal (7)

```
if ( $C1 \neq C2$ ) then
  Inserisci  $e_h$  nell'albero  $T^*$ ;  $k = k + 1$ ;
  // Unisci le componenti  $C1$  e  $C2$ 
  for  $q = 1$  to  $n$  do
    if ( $comp[q] = C2$ ) then
       $comp[q] = C1$ ;
    end if
  end for
end if
 $h = h + 1$ ;
end while
if ( $k \neq n - 1$ ) then
  il grafo  $G$  è disconnesso;
end if
```

Algoritmo di Kruskal: Esempio

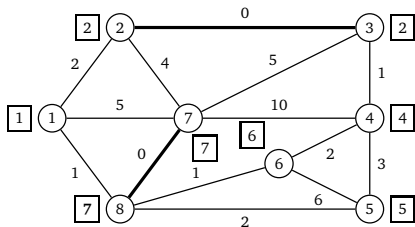


Inizializzazione delle etichette $comp[i]$

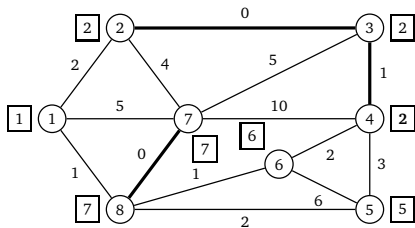


Iterazione 1:
inserimento lato $\{2, 3\}$

Algoritmo di Kruskal: Esempio (2)

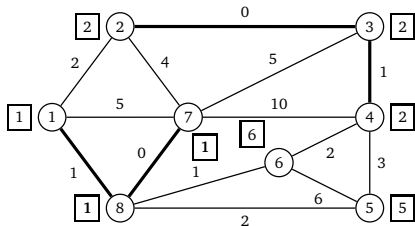


Iterazione 2:
inserimento lato $\{7, 8\}$

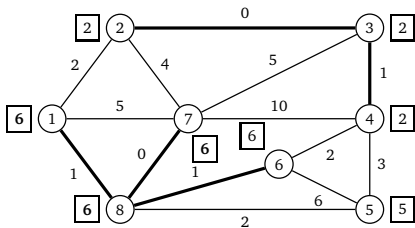


Iterazione 3:
inserimento lato $\{3, 4\}$

Algoritmo di Kruskal: Esempio (3)

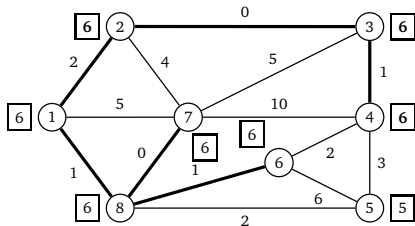


Iterazione 4:
inserimento lato $\{1, 8\}$

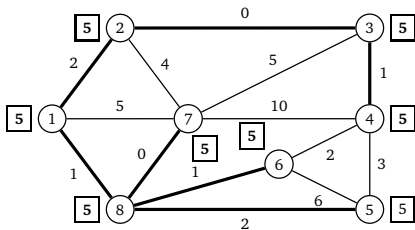


Iterazione 5:
inserimento lato $\{6, 8\}$

Algoritmo di Kruskal: Esempio (4)



Iterazione 6: il lato
 $\{6, 4\}$ chiude un ciclo,
inserimento lato $\{1, 2\}$



Iterazione 7:
inserimento lato $\{5, 8\}$

Algoritmo di Ford-Fulkerson (versione 2)

Step 1. [Inizializza]

Poni $x_{ij} = 0$, per ogni $(i, j) \in A$, e $v = 0$.

Step 2. [Determina Cammino Aumentante]

Dichiara il vertice s non espanso con etichetta $[+s, \infty]$, mentre dichiara tutti gli altri vertici non etichettati (e non espansi).

- (i) Se tutti i nodi etichettati sono già espansi, allora il flusso v è massimo (i.e, la soluzione x è ottima), quindi STOP
- (ii) Se esiste un vertice i etichettato ma non ancora espanso, provvedi a espanderlo:
 - Per ogni vertice $j \in \Gamma_i$ non etichettato per cui $x_{ij} < u_{ij}$ assegna l'etichetta $[+i, \delta_j]$, dove $\delta_j = \min\{u_{ij} - x_{ij}, \delta_i\}$;
 - Per ogni vertice $j \in \Gamma_i^{-1}$ non etichettato per cui $x_{ji} > 0$ assegna l'etichetta $[-i, \delta_j]$, dove $\delta_j = \min\{x_{ji}, \delta_i\}$.
- (iii) Se t non risulta etichettato torna a passo (i).

Algoritmo di Ford-Fulkerson (versione 2)

Step 3. [Aumenta il Flusso x]

Sia P il **cammino aumentante** dal vertice s al vertice t (ricostruito usando le etichette).

Per ogni $(i, j) \in P$:

(i) Se l'arco (i, j) è percorso nel suo verso originario (da i a j):

$$x_{ij} = x_{ij} + \delta_t;$$

(ii) Se l'arco (i, j) è percorso nel verso contrario (da j a i):

$$x_{ij} = x_{ij} - \delta_t.$$

Aggiorna il flusso $v = v + \delta_t$.

Annulla tutte etichette e ritorna allo Step 2.

NOTA: Quando l'Algoritmo di Ford-Fulkerson termina, l'insieme dei vertici etichettati S e quello dei vertici non etichettati $\bar{S}$ costituiscono un **taglio di capacità minima**.